192425192

G Vishnu Madhav

EXP29:

```
#include <stdio.h>
#include <string.h>

/* SHA-3 state: 5x5 matrix of 64-bit lanes (1600 bits).
   Block size r = 1024 bits = 16 lanes (rate part), capacity c = 576 bits = 9 lanes.
   nz[x][y] = 1 if the lane has at least one nonzero bit. */
int nz[5][5];

void show(const char *title) {
    int x, y;
    printf("%s\n", title);
    for (y = 0; y < 5; y++) {
        printf("    ");
        for (x = 0; x < 5; x++) printf("%d ", nz[x][y]);
        printf("\n");
    }
}

int zero_lanes() {
    int x, y, z = 0;
    for (x = 0; x < 5; x++) for (y = 0; y < 5; y++) if (!nz[x][y]) z++;
    return z;
}

int main() {
    int i, x, y, round = 0;
```

```
    /* P0 is XORed into the first 16 lanes (lane i is at x = i%5, y = i/5);
       capacity lanes (i = 16..24) start as zero */
    memset(nz, 0, sizeof nz);
    for (i = 0; i < 16; i++) nz[i % 5][i / 5] = 1;
    show("State after absorbing P0 (1 = lane has a nonzero bit, 0 = all-zero lane):");
    printf("Zero (capacity) lanes: %d\n\n", zero_lanes());

    while (zero_lanes() > 0) {
        int C[5], D[5], t[5][5];
        round++;
        /* theta: C[x] = XOR of column x; lane (x,y) ^= C[x-1] ^ ROT(C[x+1])
           (a lane is nonzero if it, or either column sum, is nonzero)       */
        for (x = 0; x < 5; x++) { C[x] = 0; for (y = 0; y < 5; y++) C[x] |= nz[x][y]; }
        for (x = 0; x < 5; x++) D[x] = C[(x + 4) % 5] | C[(x + 1) % 5];
        for (x = 0; x < 5; x++) for (y = 0; y < 5; y++) nz[x][y] |= D[x];
        /* rho only rotates bits inside a lane; pi moves lanes (zero lanes are tracked) */
        for (x = 0; x < 5; x++) for (y = 0; y < 5; y++) t[y][(2 * x + 3 * y) % 5] = nz[x][y];
        memcpy(nz, t, sizeof nz);
        printf("After round %d:", round);
        printf("  zero lanes left = %d\n", zero_lanes());
        show("");
    }
    printf("All capacity lanes contain at least one nonzero bit after %d round(s)\n", round);
    printf("(the theta step of the first round already diffuses every column).\n");
    return 0;
}
```

OUTPUT:

```
State after absorbing P0 (1 = lane has a nonzero bit, 0 = all-zero lane):
   1 1 1 1 1
   1 1 1 1 1
   1 1 1 1 1
   1 0 0 0 0
   0 0 0 0 0
Zero (capacity) lanes: 9

After round 1:  zero lanes left = 0

   1 1 1 1 1
   1 1 1 1 1
   1 1 1 1 1
   1 1 1 1 1
   1 1 1 1 1
All capacity lanes contain at least one nonzero bit after 1 round(s)
(the theta step of the first round already diffuses every column).
```